Searching

DSA27 — Lecture 08 · Data Structures and Algorithms

Dr. Haitham A. El-Ghareeb

Faculty of Computers and Information Sciences, Mansoura University

Week 8 · Fall 2026

Where We Are

From structures to algorithms

Weeks 4–7 built **structures**: dynamic array, linked list, stack, queue.

Weeks 8–10 are about **algorithms** on the simplest of them, the array:

- **Week 8:** find a value — **searching**
- **Weeks 9–10:** put the values in order — **sorting**

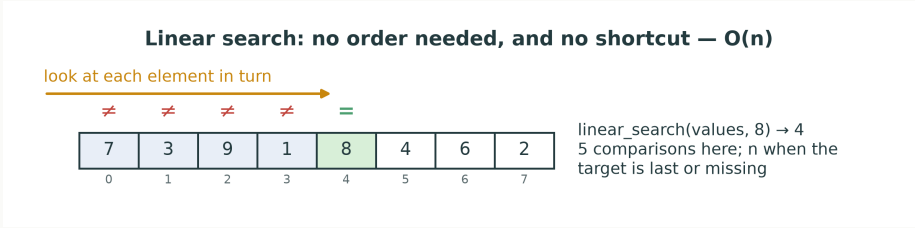
Today's question: **what does it buy you to keep an array sorted?**

Today

1. Linear search: the baseline
2. Binary search: halving — and why it is so easy to get wrong
3. `lower_bound` and `upper_bound`: positions, duplicates, insertion
4. Jump, exponential and interpolation search
5. Measured: eight functions, one picture
6. When is sorting first worth it?

Linear Search

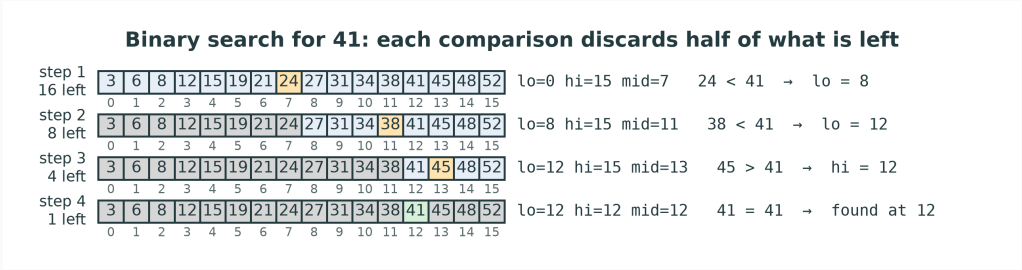
Look at everything



- Works on **any** sequence, sorted or not.
- Found at index i: i + 1 comparisons. Missing: **n**. Worst case $O(n)$.

Binary Search

Halve what is left



Compare the target with the **middle** element. Equal: found. Smaller: it can only be on the left. Larger: only on the right.

The algorithm, and the promise it keeps

Keep two indices, `lo` and `hi`: the part still in play is `values[lo..hi]`, both ends included.

1. `lo = 0`, `hi = n - 1`
2. while `lo <= hi`: `mid = (lo + hi) // 2`
   - `values[mid] == target` → return `mid`
   - `values[mid] < target` → `lo = mid + 1`
   - otherwise → `hi = mid - 1`
3. return −1

Invariant: if the target is anywhere, it is in `values[lo..hi]`.

How many comparisons?

Each comparison halves the range: after k comparisons at most $n/2^k$ remain.

n	at most $\lfloor \log_2 n \rfloor + 1$ comparisons
1,000	10
1,000,000	20
1,000,000,000	30

Binary search is $O(\log n)$ — a thousand times more data costs 10 more comparisons.

Easy to state, hard to get right

- `while lo < hi` with an inclusive `hi` → **misses** the last candidate.
- `hi = mid` with an inclusive `hi` → can **loop for ever**.
- `lo = mid` → can **loop for ever** when `hi = lo + 1`.
- `mid = (lo + hi) / 2` in Java or C → **overflows** when `lo + hi` exceeds the largest `int`.

Recursive binary search

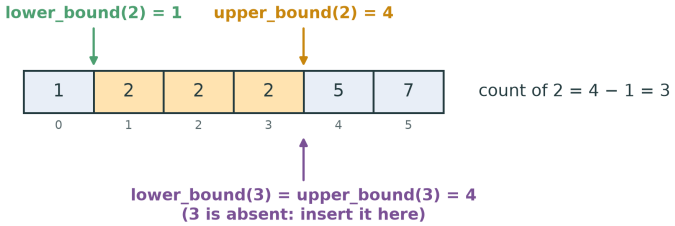
Same idea: search the half that can hold the target, **by a call**.

- $O(\log n)$ time, but also $O(\log n)$ **stack** — the loop is $O(1)$ space.
- Pass `lo` and `hi` down. **Never slice**: `values[mid + 1:]` copies — $O(n)$ per call.

Bounds: Searching for Positions

lower_bound and upper_bound

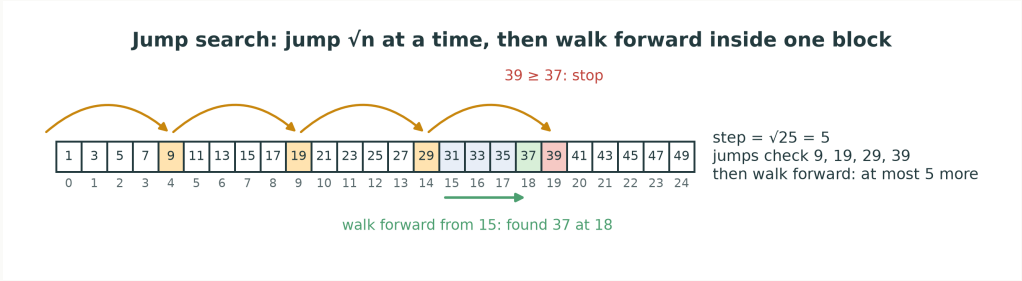
Bounds are positions between elements: where x would be inserted



- `lower_bound(v, x)`: the first index with `v[i] >= x`
- `upper_bound(v, x)`: the first index with `v[i] > x`
- Both may return **`len(v)`**. Neither returns −1.

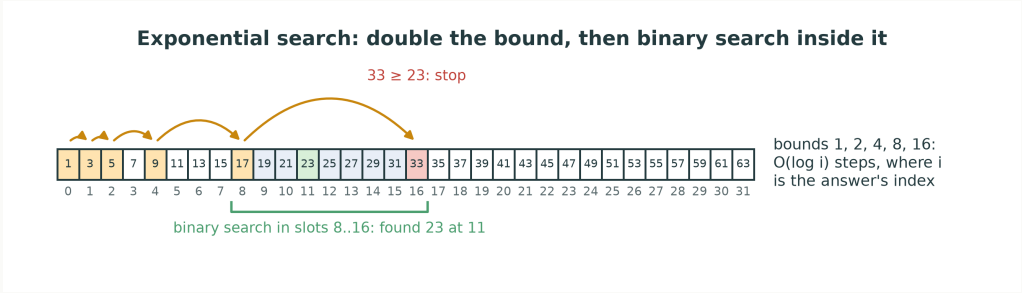
Three More Searches

Jump search: $O(\sqrt{n})$



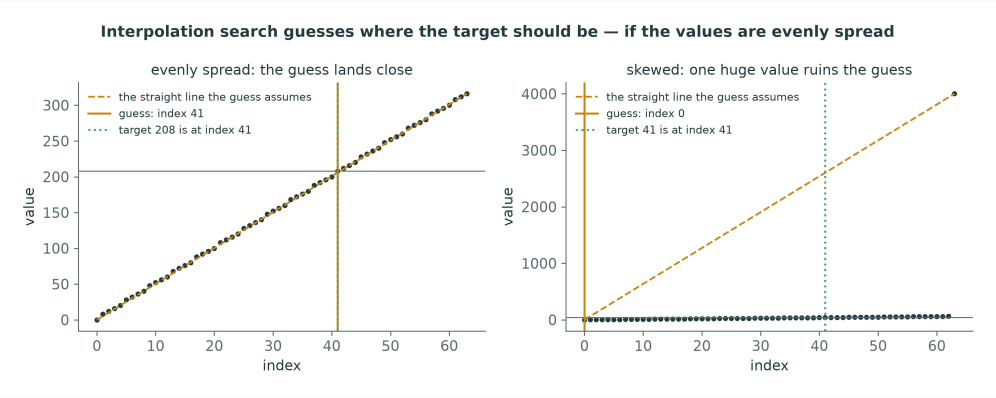
Jump ahead **√n** at a time until you pass the target; then walk forward inside that one block.

Exponential search: $O(\log i)$



Check slots 1, 2, 4, 8, … until one passes the target; then binary search between the last two bounds.

Interpolation search: guess, don’t halve

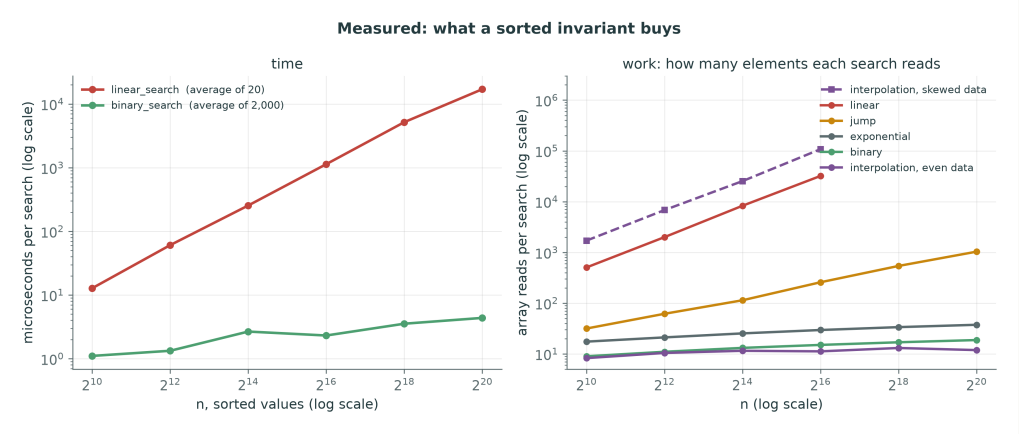


$$\text{pos} = \text{lo} + \frac{(\text{target} - v[\text{lo}])\,(\text{hi} - \text{lo})}{v[\text{hi}] - v[\text{lo}]}$$

Evenly spread values: $O(\log \log n)$ on average. Skewed: $O(n)$.

Measured

Eight functions, one picture



When is sorting first worth it?

Binary search needs sorted data, and sorting costs $O(n \log n)$ (Weeks 9–10).

k searches on n values	Cost
linear search, k times	$O(kn)$
sort once, then binary search k times	$O(n \log n + k \log n)$

Sorting wins once **k is larger than about** $\log n$ — for a million values, after a few dozen searches.

The eight functions

Function	Needs	Time	Note
`linear_search`	nothing	$O(n)$	first match
`binary_search`	sorted	$O(\log n)$	loop, $O(1)$ space
`binary_search_recursive`	sorted	$O(\log n)$	$O(\log n)$ stack; never slice
`lower_bound`	sorted	$O(\log n)$	first `v[i] >= x`; may be `len`
`upper_bound`	sorted	$O(\log n)$	first `v[i] > x`; may be `len`
`jump_search`	sorted	$O(\sqrt{n})$	forward only
`exponential_search`	sorted	$O(\log i)$	unknown length; near the front
`interpolation_search`	sorted, even	$O(\log \log n)$ avg, $O(n)$ worst	guesses

This Week

Exercises: `dsa/searching.py`

Function	The trap
`linear_search`	unsorted input; return the **first** match
`binary_search`	`lo <= hi`; `mid ± 1`; –1 when missing
`binary_search_recursive`	pass `lo`, `hi`; **no slices**
`lower_bound`, `upper_bound`	half-open `[lo, hi)`; `>=` against `>`
`jump_search`	`math.isqrt`; the last, partial block
`exponential_search`	slot 0 first; bound capped at `len - 1`
`interpolation_search`	equal ends; target outside `[v[lo], v[hi]]`

```
pytest tests/test_searching.py -v
```

Homework 8 — before Lecture 09

1. **Implement** `dsa/searching.py` until all forty tests pass.
2. **Trace** `binary_search` on `[2, 5, 8, 12, 16, 23, 38, 56, 72, 91]` for 23 and for 60, as tables of `lo`, `hi`, `mid`.
3. **Measure** linear against binary search in `notebooks/08-searching.ipynb`, and count reads with a counting sequence, as in the figure.
4. **Break it.** Change `lo <= hi` to `lo < hi` in your `binary_search`. Which tests fail, and why exactly those?

The midterm window

Weeks 1–7 are examinable. For each week, can you:

- **1–2:** define an ADT; give Θ of a loop; prove a Big-O bound?
- **3:** write a recurrence and trace a call stack?
- **4:** explain amortised $O(1)$ append?
- **5–7:** draw a linked list, stack, queue or ring **after every operation**?

The mock exam for weeks 1–7 is in the question bank.

Practise online

- **Week 8 quiz**: 22 multiple-choice questions, every answer explained
- **Written practice** on the same page: 5 questions. Write, then mark yourself against the marking points
- In Arabic: week 8

Train for the midterm online

The midterm simulator draws a new paper from the bank every time:

- Full midterm — 60 questions, 90 min
- Exam format — 30 questions + 30 marks written, 2 h
- Half paper — 30 questions, 45 min
- Explain-level challenge — 25 “why” questions
- My weak spots · My mistakes
- One lecture, 20 questions: 1 · 2 · 3 · 4 · 5 · 6 · 7

On paper: mock exam, weeks 1–7 and mock exam B, with its own answers.

Summary

Seven things to keep

1. Linear search needs no order and costs $O(n)$.
2. Binary search needs **sorted** data and costs $O(\log n)$: 20 comparisons for a million.
3. Its invariant: if the target is anywhere, it is in `values[lo..hi]`.
4. Get the boundaries right: `lo <= hi`, `mid ± 1` — test sizes 0, 1, 2.
5. Recursive: pass `lo` and `hi`; a slice turns $O(\log n)$ into $O(n)$.
6. `lower_bound`/`upper_bound` return **positions**: insertion points and counts of duplicates.
7. Jump $O(\sqrt{n})$, exponential $O(\log i)$, interpolation $O(\log \log n)$ average but $O(n)$ **worst**.

Next

Week 9 — Basic sorting. Bubble, selection and insertion sort — and counting sort, which beats $O(n \log n)$ by never comparing at all.